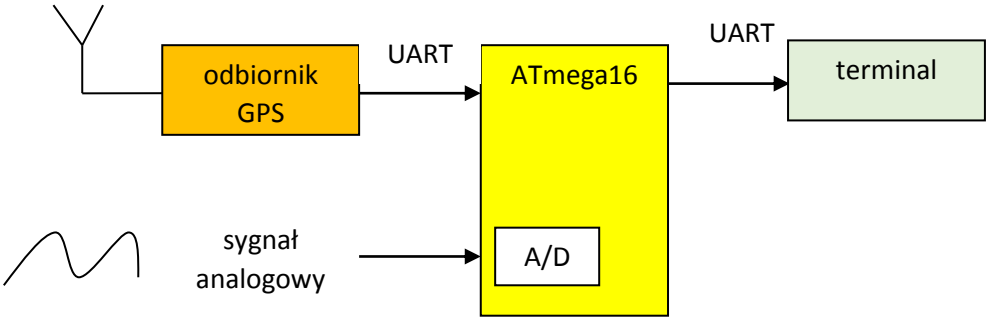
1. Zagadnienia

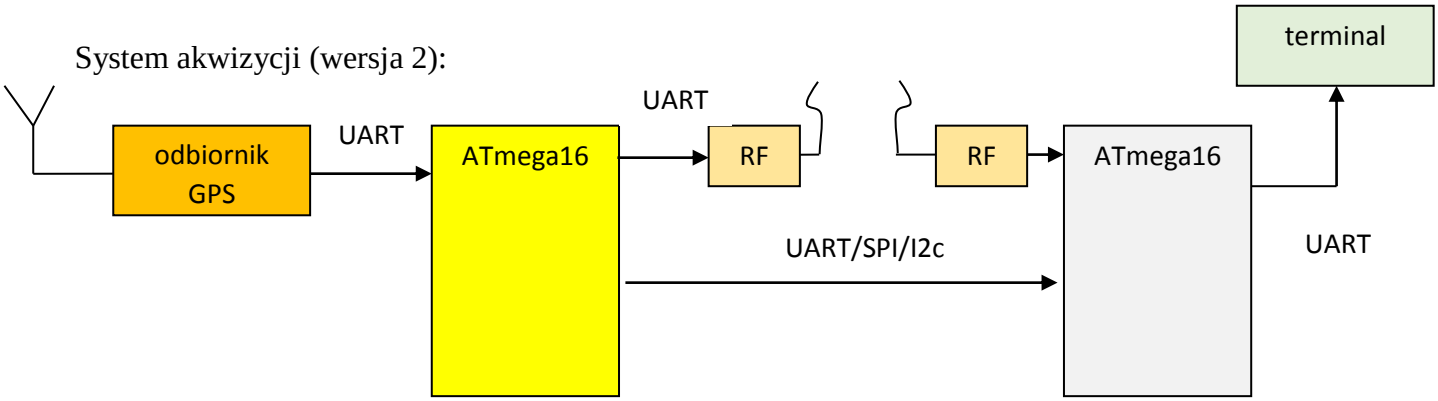
- Rozproszony system akwizycji danych - założenia projektowanego systemu
- Asembler mikrokontrolerów AVR - powtórzenie wiadomości
- Organizacja kodu programu w pamięci programu
- Makra i podprogramy
- Programowy i sprzętowy UART

2. System akwizycji danych - przykład realizowany na zajęciach

System akwizycji (wersja 1):



System akwizycji (wersja 2):



Zadania realizowane przez system:

- odbiór danych GPS (co 1 sek) - UART (sprzętowy),
- wyodrębnienie informacji o czasie,
- pomiar sygnału analogowego (np. napięcia), obsługa przetwornika A/D,
- powiązanie informacji o czasie i wartości napięcia,
- zapis danych do pamięci danych (SRAM),
- przesłanie danych do terminala - UART (programowy lub sprzętowy),

- przesłanie danych do podsystemu/ów (ATmega16),
- komunikacja przewodowa (UART,SPI,I2C),
- komunikacja bezprzewodowa (RF, interfejs optyczny)

3. Asembler mikrokontrolerów AVR, powtórzenie wiadomości

- Utwórz nowy projekt (***Project > Project Wizard***) według wskazówek podanych we wprowadzeniu do laboratorium
- Wpisz kod przykładowego programu: Odejmowanie liczb: 50 - 20 =

Program_01

```
.nolist                    // komentarz
.include "m16def.inc" ; komentarz
.list
.listmac
.device ATmega16

.cseg
.org 0x0000

jmp 0x0030 ;skok do adresu 0x0030
// jmp START
// wektory przerwań

.org 0x0030
START:
// odejmowanie
ldi r16, 50
ldi r17, 20
sub r16, r17

nop
jmp PC

//LOOP_END:
//jmp LOOP_END
```

Taki nagłówek standardowo będzie rozpoczynał każdy program

Organizacja kodu w pamięci programu

- Sprawdź działanie programu w pracy krokowej, poprawność skoku i poprawność otrzymanego wyniku odejmowania. Dobierz liczby tak, aby uzyskać wynik ujemny (np. 100 - 250 =). Obserwuj zachowanie znacznika C w rejestrze SREG.
- Dobierz liczby tak, aby wynik odejmowania był równy zero (np. 10–10=). Obserwuj zachowanie znacznika Z.

W każdym programie na początku umieszczamy nagłówek złożony z kilku dyrektyw, które przygotowują środowisko programistyczne do pracy z wybranym typem procesora (ATmega16). Następnie określamy, że nasz kod docelowo powinien znaleźć się w pamięci programu (**.cseg**).

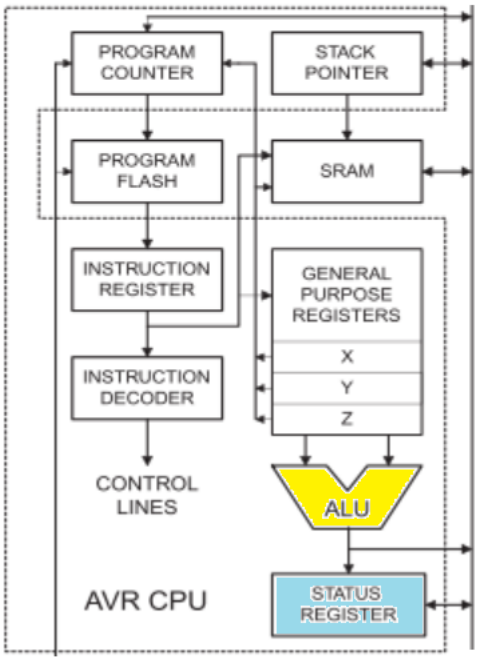
Organizujemy rozmieszczenie kodu korzystając z dyrektyw **.org**. Pierwsza wykonywana instrukcja skoku **JMP 0x0030** umieszczona jest w komórce o adresie 0x0000, osiągamy to za pomocą dyrektywy **.org 0x0000**. Wykonując instrukcję skoku **JMP 0x0030** procesor "skacze" do adresu 0x0030, zatem tam powinny zostać umieszczone kolejne instrukcje naszego programu, uzyskujemy to stosując dyrektywę **.org 0x0030**.

W programie zastosowano etykiety. Etykieta to nazwa (ciąg liter lub cyfr - nazwa etykiety nie może zaczynać się cyfrą!) zakończona znakiem dwukropka (:). Wielkość liter nie ma znaczenia - START, start, STart - to ta sama etykieta. W programie zadeklarowano etykiety START, INC_REG, LOOP_END.

Dzięki zastosowaniu etykiet nie musimy podawać (i pamiętać) bezpośrednio wartości liczbowych adresów do których ma zostać wykonany skok (np. JMP 0x0030), środowisko AVR Studio automatycznie przypisze wartości liczbowe do odpowiednich etykiet.

4. ALU (jednostka-arytmetyczno logiczna)

Jednostka arytmetyczno-logiczna ALU (*Arthmetic Logic Unit*) jest najważniejszym podzespołem, w którym wykonywane są wszystkie operacje arytmetyczne i logiczne. ATmega16 jest mikrokontrolerem 8-bitowym - to znaczy, że jednostka ALU wykonuje obliczenia na słowie 8-bitowym.



5. SREG (rejestr znaczników, rejestr flag)

W programowaniu procesorów bardzo często wykorzystujemy pojęcie rejestru. Rejestr to komórka pamięci (układu cyfrowego z przerzutnikami) przeznaczona do zapamiętania informacji). W przypadku omawianego mikrokontrolera ATmega16 najczęściej mówimy o rejestrach 8 i 16 - bitowych. Ze względu na zastosowania możemy wyróżnić:

- rejestry danych – do przechowywania danych (argumentów, wyników obliczeń)
- rejestry adresowe – do przechowywania adresów
- rejestry ogólnego zastosowania (ang. *general purpose*) - będące połączeniem dwóch powyższych typów, czyli mogące przechowywać zarówno dane, jak i adresy,
- rejestry specjalne - określające stan wykonania, wśród nich wymienić można rejestr PC (wskaźnik instrukcji), SP (wskaźnik stosu), rejestr flag (SREG),

Rejestr SREG (*Status Register* - rejestr znaczników) jest specjalnym rejestrem do którego wpisywane są informacje o przebiegu **ostatnio wykonanej** operacji w ALU (np. było przeniesienie, nadmiar, wynik zerowy itp.). Rejestr SREG składa się z 8 bitów:

7	6	5	4	3	2	1	0
I	T	H	S	V	N	Z	C

6. Instrukcje warunkowe, instrukcje skoku warunkowego

Informacje o stanie procesora zawarte w rejestrze SREG, są wykorzystywane w instrukcjach skoków warunkowych do tworzenia pętli programowych (warunek pozostania w pętli lub wyjścia z pętli). Mikrokontrolery AVR posiadają kilkanaście różnorodnych instrukcji skoków warunkowych (zobacz: ***Help ->Assembler Help-> Branch Instructions***)

- Utwórz nowy projekt (***Project -> Project Wizard***).
- Wpisz kod programu: Odliczanie do zera

Program_02

```
.nolist                  // komentarz
.include "m16def.inc" ; komentarz
.list
.listmac
.device ATmega16

.cseg
.org 0x0000

jmp START   ;skok do etykiety
            ;START (adres 0x0030)

// wektory przerwań

.org 0x0030
START:
// odliczanie w pętli
ldi r16, 10

Loop_Count:
dec r16
brne Loop_Count

nop

jmp PC
```

nagłówek

Organizacja kodu w pamięci programu

Wpisanie początkowej wartości do r16

warunek instrukcji BRNE jestspełniony

Zmniejszenie r16 o 1 i sprawdzenie znacznika Z, jeżeli:
Z = 0 (wartość r16 różna od zera) - to skok do etykiety Loop_Count
Z = 1 (wartość r16 osiągnęła zero) - to przejście do następnej instrukcji (NOP)

warunek instrukcji BRNE nie jest spełniony

Instrukcje sprawdzające znacznik Z i wykonujące skok, to np. BRNE lub BREQ. W przykładzie zastosowano BRNE (można też zastosować BREQ - oczywiście konstrukcja programu pętli będzie trochę inna). Działanie instrukcji BRNE:

```
If Rd ≠ Rr (Z = 0) then PC ← PC + k + 1, else PC ← PC + 1
```

Jeżeli po wykonaniu instrukcji dekrementacji w liczniku r16 jest liczba różna od zera, to znacznik Z nie zostanie ustawiony (Z=0), jeżeli r16 osiągnął zero to znacznik Z zostanie ustawiony na 1 (Z=1).

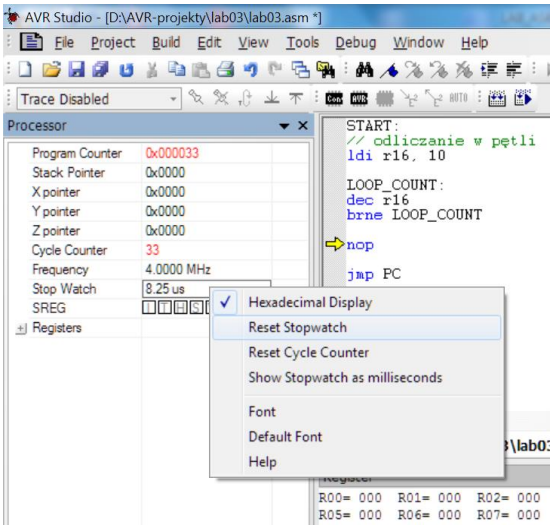
Działanie instrukcja BREQ:

```
If Rd = Rr (Z = 1) then PC <- PC + k + 1, else PC <- PC + 1

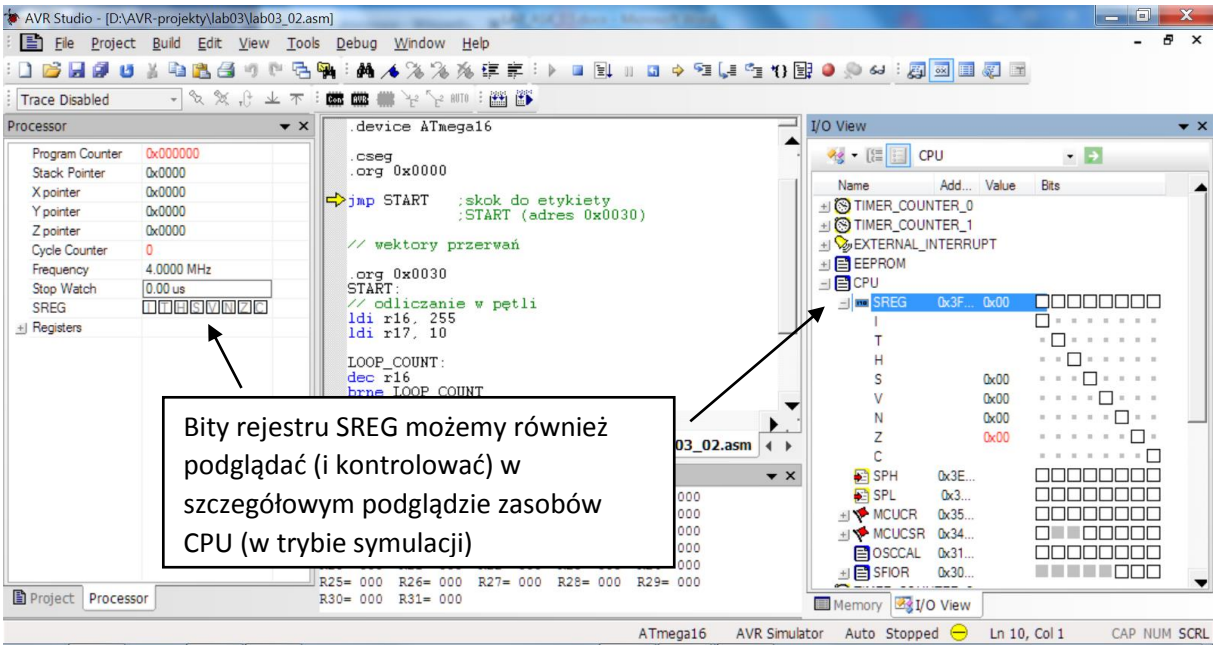
-64 ≤ k ≤ +63
```

Liczba k (liczba dodawana do aktualnej wartości PC - zakres skoku) może przyjmować zarówno wartości dodatnie, jak i ujemne, zatem w naszym programie możemy wykonać skok do "przodu" i do "tyłu" (niestety dla instrukcji BREQ i BRNE zakres skoku jest ograniczony +/- 64 komórki, nie możemy zatem wykonać skoku w dowolne miejsce obszaru pamięci).

- Sprawdź działanie programu w pracy krokowej, "zmierz" czas opóźnienia wprowadzanego przez pętlę (dla domyślnie ustawionej częstotliwości taktowania procesora 4MHz).
- Sprawdź jakie maksymalne opóźnienie można uzyskać w pętli (wpisz do r16 wartość 255).

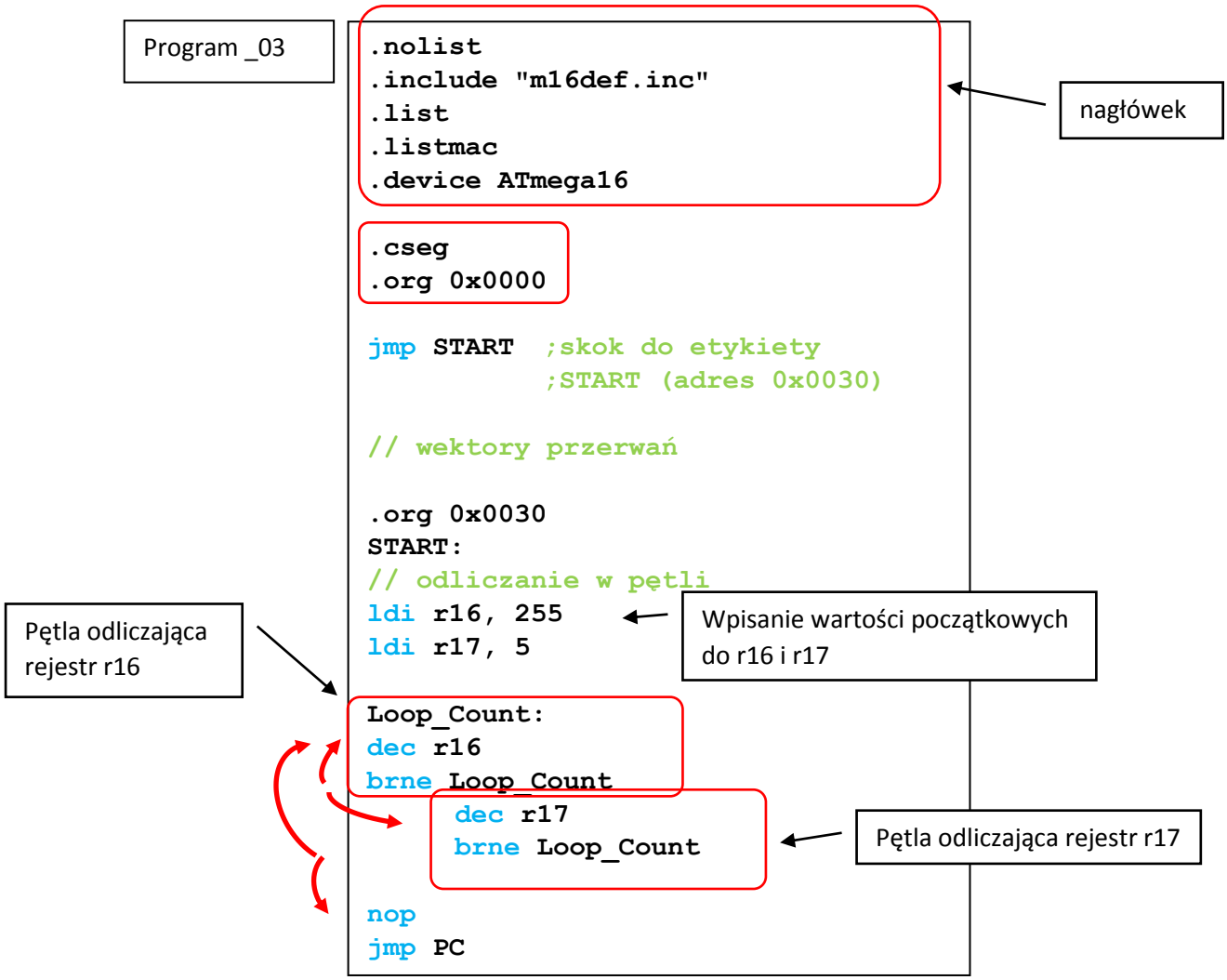


Podczas symulacji, w dowolnym momencie możemy "wyzerować" licznik wskazujący liczbę cykli zegara (Cycle Counter) oraz odmierzony czas (Stop Watch) (***Prawy-Przycisk-Myszy***). Wyzeruj wskazania na początku pętli **Loop_Count** i odczytaj po wyjściu z pętli.



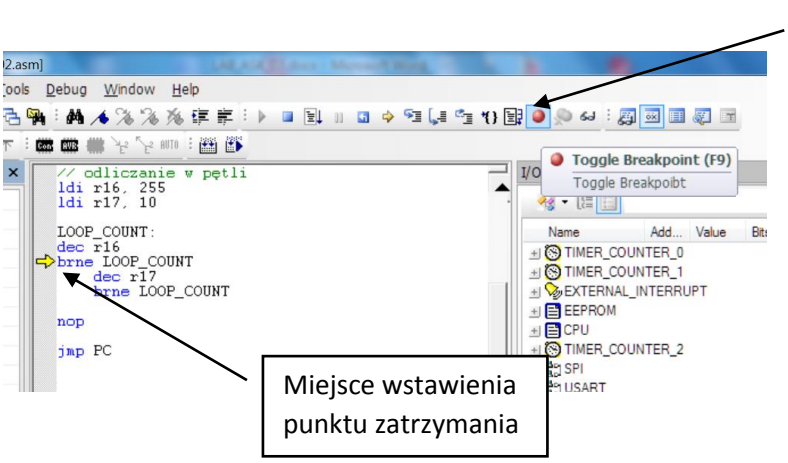
7. Pętla w pętli

Wykorzystując do odliczania rejestr 8-bitowy oraz pojedynczą pętlę, możemy generować opóźnienia rzędu mikrosekund, aby wydłużyć opóźnienia możemy zastosować pętle w pętli.



W przykładzie po odliczeniu do zera rejestru r16, program przechodzi do instrukcji DEC r17, zmniejsza zawartość rejestru r17, sprawdza warunek i jeżeli nie doliczył do zera to ... wraca do Loop_Count, czyli zaczyna odliczać r16. Mamy zatem zagnieżdżenie pętli. Całkowity czas opóźnienia będzie zatem wielokrotnością (liczba w r17) i czasu wykonania podstawowe pętli odliczającej r16.

- Sprawdź działanie programu, "zmierz" czas opóźnienia wprowadzanego przez pętlę dla wartości r16 =255, r17=10).
- Sprawdź jakie maksymalne opóźnienie można uzyskać w pętli (wpisz do r16 i do r17 wartość 255).



Analizując działanie pętli w pracy krokowej warto zastosować punkty zatrzymania (Breakpoint).

Breakpoint zostanie wstawiony w aktualnie wskazywanym w programie w miejscu.

Po wstawieniu punktów zatrzymania możemy uruchomić symulację w trybie pracy ciągłej używając poleceń: ***Debug -> Auto-Step (Alt+F5)*** lub ***Debug -> Run (F5)***. Przerwanie pracy ciągłej**:** ***Debug -> Break (Ctrl+F5)*** lub ***Debug -> Reset (Shift+F5).***

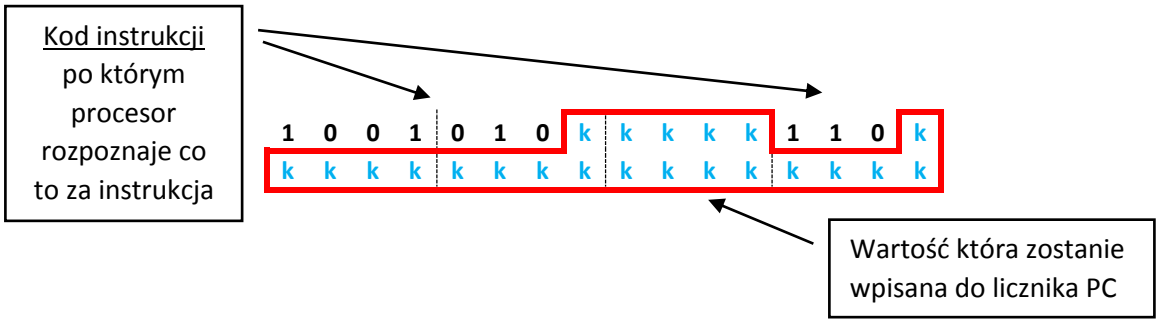
8. Instrukcja skoku bezwarunkowego

Co to znaczy, że instrukcja **JMP** wykonuje skok (przekierowanie) do wskazanego adresu? W pomocy (***Help > Assembler Help***) możemy zapoznać się z opisem instrukcji i przeanalizować działanie skoku.

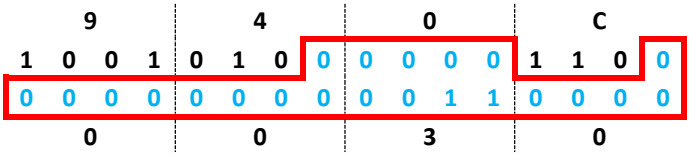
Działanie instrukcji JMP polega na bezpośrednim wpisie wartości (liczby k)do licznika (rejestru) Program Counter (PC). **Program Counter** jest najważniejszym rejestrem w każdym procesorze. Jego aktualna wartość wskazuje bezpośrednio adres w pamięci programu z którego zostanie pobrana kolejna instrukcja do wykonania.

Analiza struktury instrukcji skoku

Pamięć programu mikrokontrolera ATmega16 zorganizowana jest w 16-bitowe słowa (2 bajty). Jedna "komórka" pamięci to jedno słowo. Instrukcja JMP składa się z 32 bitów (4 bajty). W pamięci programu umieszczona zostanie w dwóch kolejnych komórkach. Instrukcję JMP można przedstawić w następujący sposób:



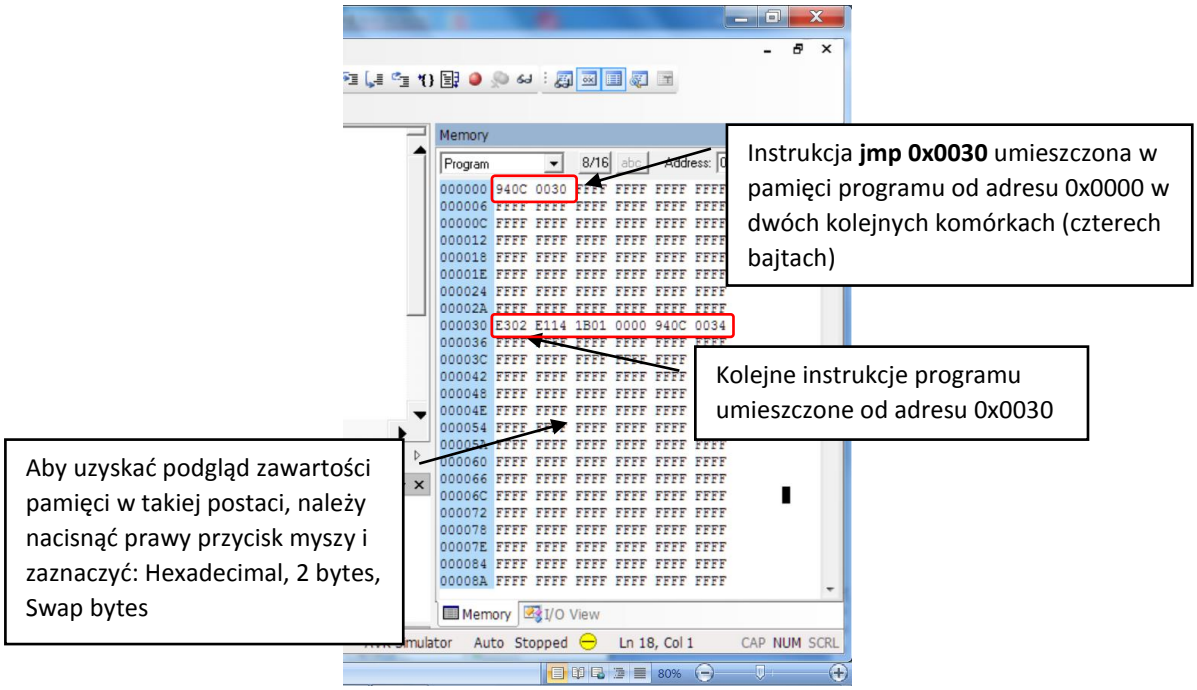
Cała instrukcja składa się z kodu instrukcji i argumentu (liczby k), który określa do jakiego adresu ma zostać wykonany skok. Przykładowo, jeżeli chcemy wykonać bezwarunkowy skok do adresu 0x0030 to musimy przyjąć wartość k = $30_{(HEX)}$:



$30_{(HEX)} = 00110000_{(2)}$

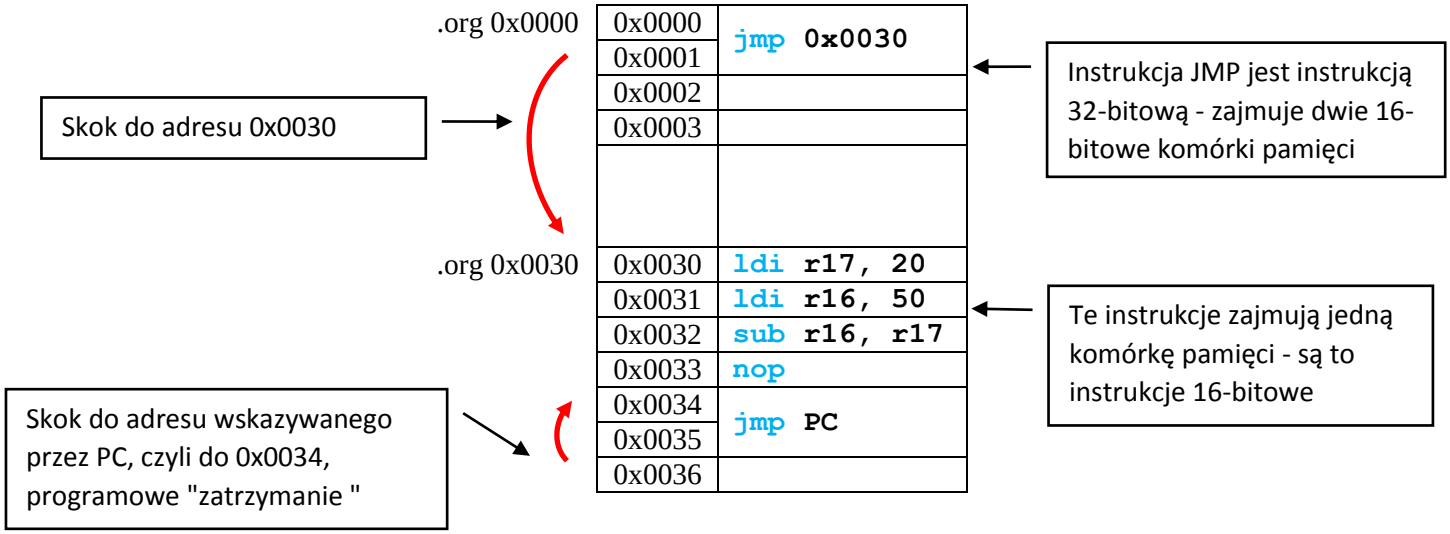
Zatem instrukcja **JMP 0x0030** przedstawiona w kodzie maszynowym to **940C0030.**W instrukcji JMP do zapisu liczby k przeznaczono 22 bity - zatem przestrzeń adresowa do której możemy wykonać skok z wykorzystaniem instrukcji JMP jest całkiem spora, zawiera się między 0 a 2^{22}, czyli 4MB.

- Po skompilowaniu(***F7***) programu i uruchomieniu trybu symulatora - otwórz podgląd pamięci (***View > Memory***), wybierz pamięć programu (Program). W pamięci programu możemy odnaleźć instrukcję **JMP 0x0030**.



Oczywiście - programując w asemblerze - nie będziemy tak szczegółowo analizowali każdej instrukcji. Warto jednak mieć pewną wiedzę o tym, jak nasz program wygląda w kodzie maszynowym i gdzie jest lokowany w pamięci programu.

Przykład obrazujący rozmieszczenie instrukcji programu w pamięci programu:

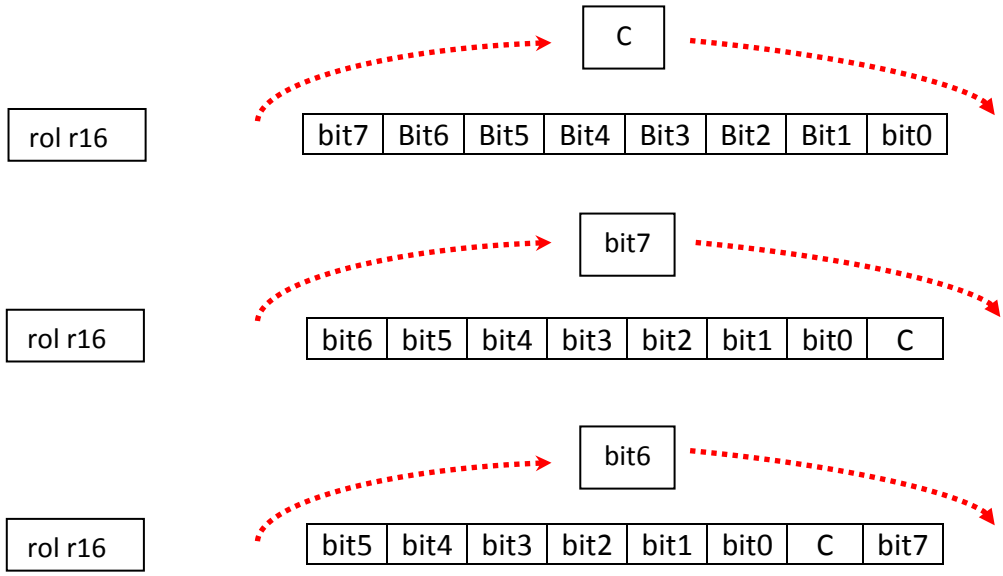


Oprócz instrukcji JMP, lista rozkazów procesorów AVR zawiera też inne instrukcje skoku: RJMP (*Relative Jump*) i IJMP (*Indirect Jump*). Ich działanie nieznacznie różni się od działania instrukcji JMP, ale wszystkie one służą do wykonania skoku bezwarunkowego.

9. Instrukcje przesuwania zawartości rejestru

Zawartość rejestru możemy przesuwać w lewo lub w prawo używając instrukcji ROL (*Rotate Left through Carry*) lub ROR (*Rotate Right through Carry*):

- Napisz program który w pętli będzie przesuwał zawartość rejestru r16 w lewo (ROL). Sprawdź działanie w pracy krokowej, obserwuj stan znacznika C.



itd ...

- Podobnie sprawdź działanie instrukcji ROR.

Oprócz instrukcji ROL i ROR zawartość rejestru możemy przesuwać za pomocą instrukcji LSR (*Logical Shift Right*) i LSL (*Logical Shift Left*):

- Napisz program który porówna działanie instrukcji ROL i LSL oraz ROR i LSR. Jaka jest różnica w działaniu tych instrukcji. Można zauważyć, że logiczne przesuwanie w lewo (LSL) o jedną pozycję - to mnożenie przez 2, a logiczne przesuwanie w prawo o jedną pozycję (LSR) - to dzielenie przez 2.

10. Makra i podprogramy

Makro - to struktura (pre)definiująca fragment programu, który może zawierać zmieniające się parametry. Wywołanie makra (jego nazwy i parametrów) w programie głównym, powoduje że translator uwzględnia treść makra w miejscu jego wywołania. Makra w asemblerze definiuje się na początku programu.

Treść makra umieszczamy pomiędzy dyrektywami .MACRO i .ENDMACRO (lub .ENDM)

Program_04

```
.nolist    // makro
.include "m16def.inc"
.list
.listmac
.device ATmega16

.macro DODAWANIE
add r16, r17
.endmacro

.cseg
.org 0x0000

jmp START  ;skok do etykiety
           ;START (adres 0x0030)
// wektory przerwań
.org 0x0030
START:
ldi r16, 10
ldi r17, 20
DODAWANIE

ldi r16, 30
ldi r17, 40
DODAWANIE

jmp PC
```

definicja makra zawsze na początku programu

makro DODAWANIE zawiera jedną instrukcję **add r16, r17**, ale instrukcji może być więcej

wywołanie makra

10+20=30

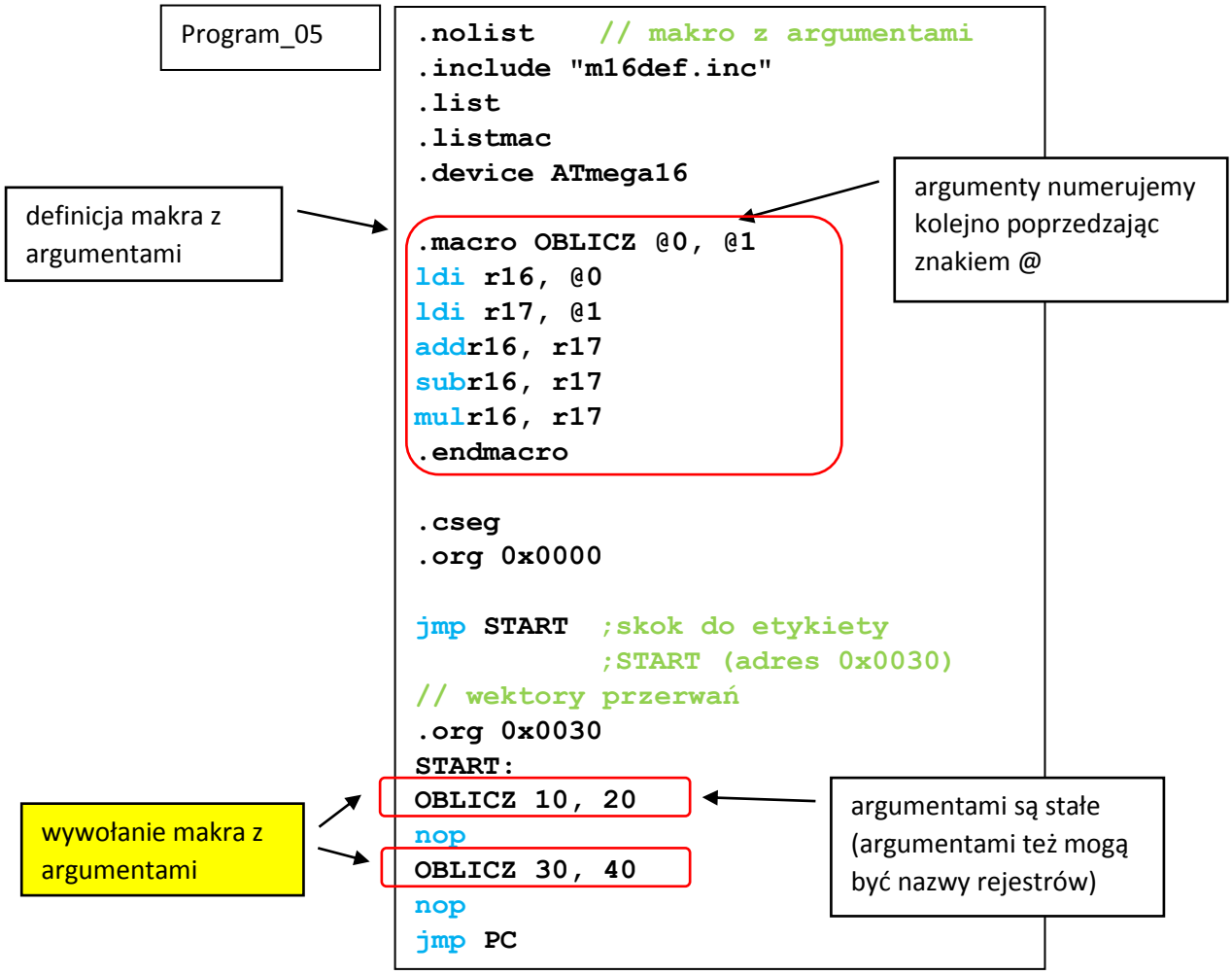
wywołanie makra

30+40=70

Na początku zdefiniowano makro DODAWANIE, w programie głównym makro "wywołano" dwukrotnie, za każdym razem zmieniając wartości składników (w rejestrach r16 i r17).
Ten typ makra nazywany jest makrem bez argumentów.

11. Makro z argumentami

Treść makra umieszczamy pomiędzy dyrektywami .MACRO i .ENDMACRO (lub .ENDM), po nazwie makra podajemy listę argumentów.



Aby wywołać zdefiniowane wcześniej makro, podajemy jego nazwę i parametry (oddzielone przecinkami).

Uruchom programy w pracy krokowej. Podczas symulacji włącz podgląd deasemblacji (***View > Disassembler***). Obserwuj jaki jest efekt wstawianie makra do kodu programu.

12. Podprogramy

Podprogram - jest wydzieloną częścią programu, do której możemy się odwoływać wielokrotnie. W odróżnieniu od makra, treść podprogramu w kodzie programu umieszczona jest raz (treść makra umieszczana jest w programie za każdym razem, gdy odwołujemy się do makra).

Wywołanie podprogramu CALL, powrót z podprogramu RET

Program_06

```
.nolist    // podprogram
.include "m16def.inc"
.list
.listmac
.device ATmega16

.cseg
.org 0x0000

jmp START  ;skok do etykiety
           ;START (adres 0x0030)
// wektory przerwań
.org 0x0030
START:
ldi r16, 10
ldi r17, 20
call DODAJ

ldi r16, 50
ldi r17, 60
call DODAJ

jmp PC
//---koniec głównego programu---

// --- podprogram ---
.org 0x0100
DODAJ:
add r16, r17
sub r16, r17
mul r16, r17
ret
```

wywołanie podprogramu

Podprogram DODAJ został umieszczony w pamięci pod adresem 0x0100 (poza głównym programem)

powrót z podprogramu

Instrukcja CALL "wywołuje" podprogram DODAJ, instrukcja RET wraca do programu głównego. Uruchom program w pracy krokowej i zastanów się dlaczego program nie działa prawidłowo?

13. Stos i jego znaczenie, instrukcje CALL i RET

Napisany program nie będzie działał prawidłowo, ponieważ w naszym programie nie zdefiniowano stosu. Stos jest wydzieloną pamięci danych w której zapisywane są informacje o adresach powrotu przy wykonywaniu skoków do podprogramu.

Jak działa instrukcja CALL i RET ?

CALL	k	Call Subroutine	STACK = PC+2, PC = k

Wywołanie instrukcji CALL powoduje podstawienie pod PC (Program Counter) wartości przypisanej do nazwy wywoływanego podprogramu. W naszym przypadku jest to nazwa DODAJ pod adresem 0x0100, czyli do PC zostanie wpisana wartość 0x0100. Działanie programu zostanie "przekierowane" do adresu 0x0100, czyli do naszego podprogramu. Jednocześnie na stos zostanie zapisany adres powrotu (a dokładnie adres kolejnej instrukcji w programie głównym po CALL). Wskaźnik stosu SP (Stack Pointer) zostaje zmniejszony i wskazuje kolejną komórkę w pamięci danych (procesor jest gotowy do zapisania kolejnej wartości na stos, np. w przypadku wykonanie kolejnego skoku do podprogramu).

RET	None	Subroutine Return	PC = STACK

Ostatnią instrukcją podprogramu jest RET. Do PC zapisywana jest wartość zapisana wcześniej (odłożona) na stosie (PC <- STACK), czyli wracamy do programu głównego (a ściślej do następnej instrukcji po wywołaniu CALL), następnie zwiększany jest wskaźnik stosu (zdejmujemy wartość ze stosu).

Definicja stosu w programie (fragment):

Program_07

```
//  ~ poprzednia część programu

.cseg
.org 0x0000

jmp START  ;skok do etykiety
           ;START (adres 0x0030)
// wektory przerwań
.org 0x0030
START:
ldi r16, 0x5F
out SPL, r16
ldi r17, 0x04
out SPH, r17

// ~ dalsza część programu
```

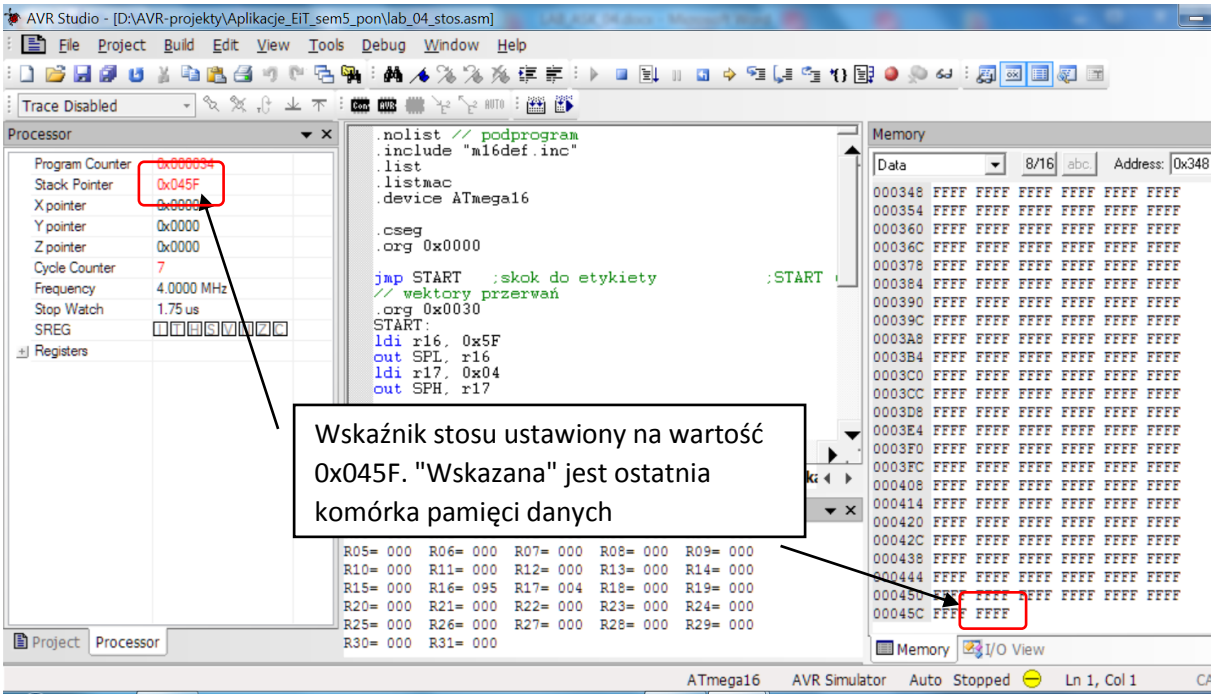
definicja stosu

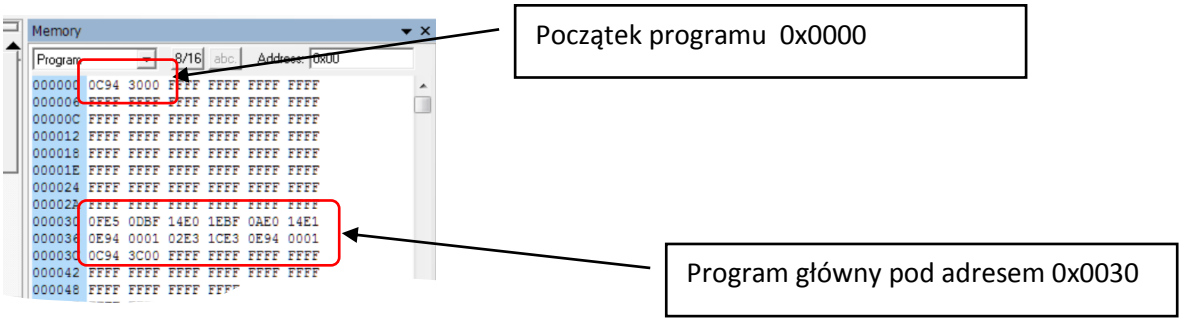
Możemy również napisać:

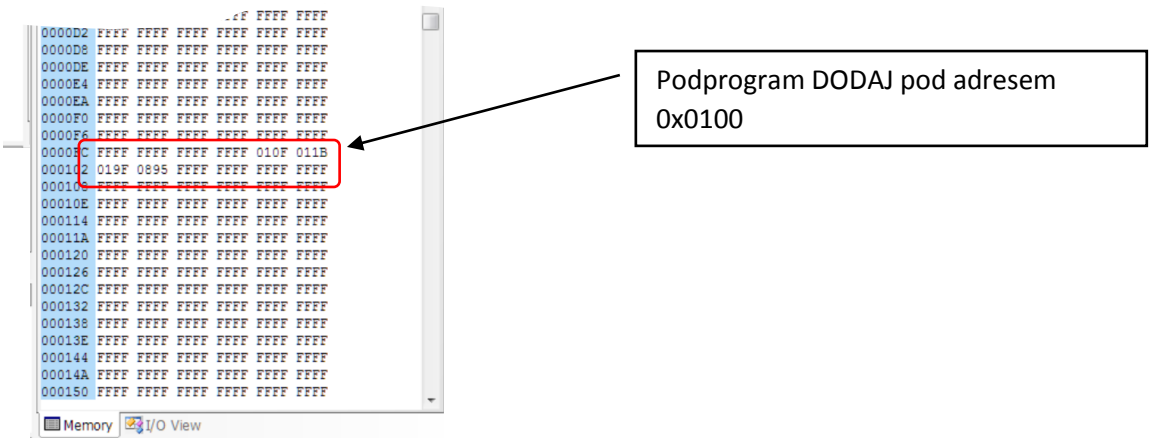
```
ldi r16, LOW(RAMEND)
out SPL, r16
ldi r17, HIGH(RAMEND)
out SPH, r17
```

Stos jest najważniejszą strukturą w procesorze lokalizowaną przez programistę, Umieszczamy go (zazwyczaj) na końcu pamięci programu i budujemy ... w górę.

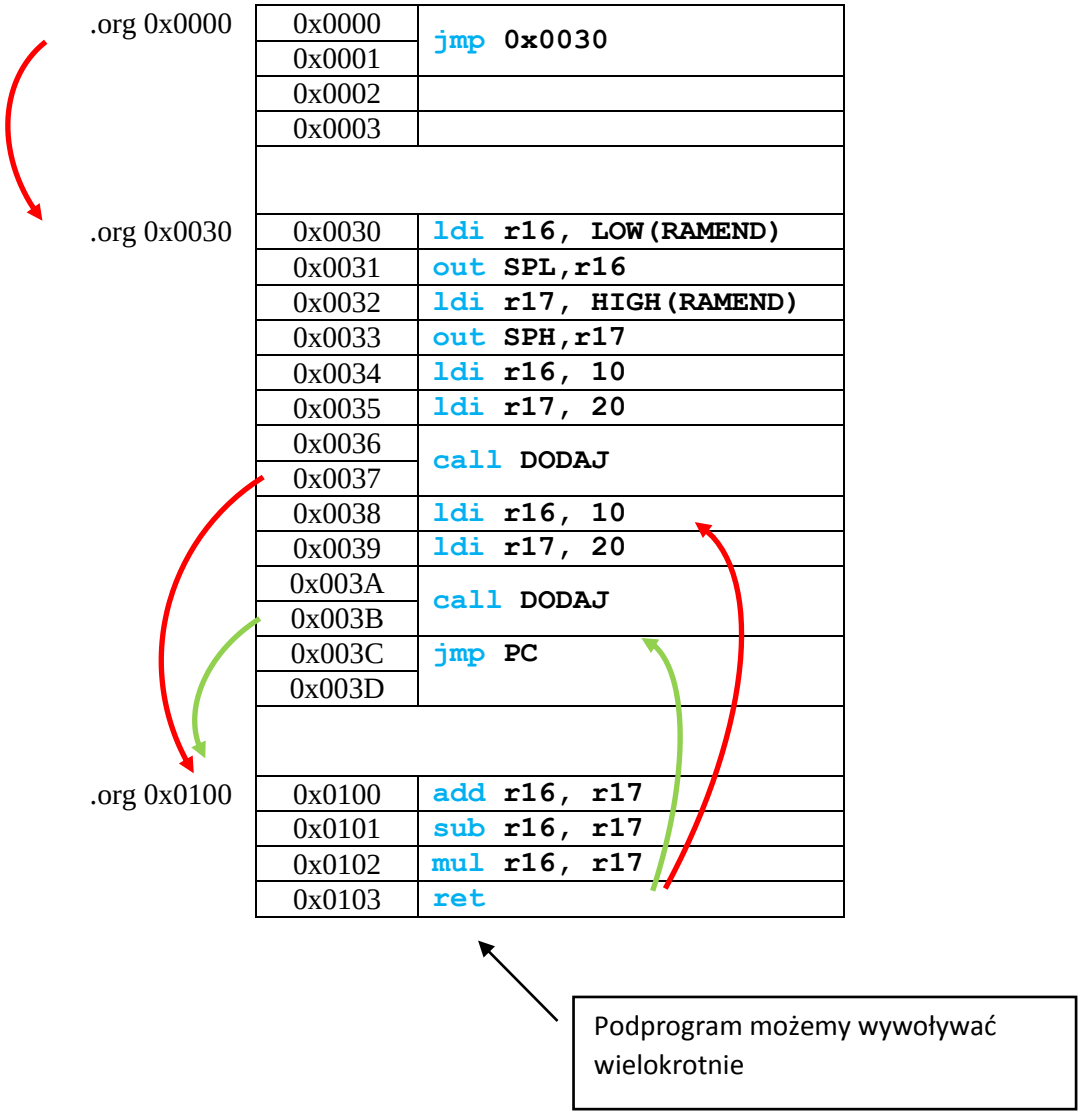
W ATmega16 ostatnia komórka pamięci danych to 0x045F lub RAMEND. Instrukcja LDI r16, LOW(RAMEND) wydziela młodszą część adresu 0x045F (czyli 5F) i umieszcza w rejestrze r16, instrukcja OUT SPL, r16 zapisuje wartość r16 do młodszej części wskaźnika stosu SP (SPL). Instrukcja LDI r17, HIGH(RAMEND) wydziela starszą część adresu 0x045F (czyli 04) i umieszcza w rejestrze r17, instrukcja OUT SPH, r17 zapisuje wartość r17 do starszej części wskaźnika stosu SP (SPH).







Rozmieszczenie instrukcji programu i podprogramu można przedstawić obrazując mapę pamięci programu:

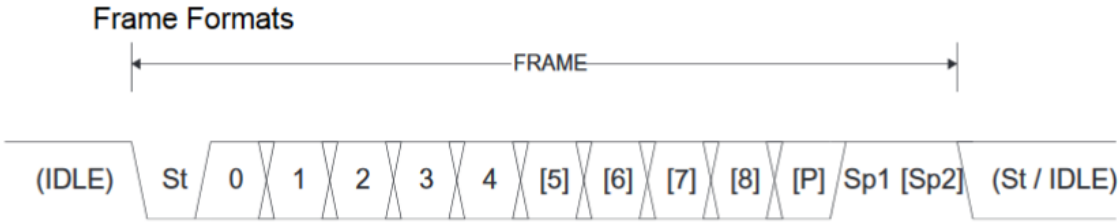


Uruchom program w pracy krokowej. Obserwuj, jak działa odkładanie adresu na stosie, obserwuj wskaźnik stosu SP i zawartość stosu (ostatnie komórki pamięci danych).

14. Programowy nadajnik UART

Napisać program realizujący programowy UART - wysyłanie zawartości rejestru (bajtu) zgodnie ze poniższymi parametrami:

9600, 8, N, 1



St	Start bit, always low.
(n)	Data bits (0 to 8).
P	Parity bit. Can be odd or even.
Sp	Stop bit, always high.

15. Sprzętowy nadajnik/odbiornik UART

Napisać program realizujący sprzętowy UART - uwzględnić ustawienia konieczne do realizacji odbioru i wysyłania znaku.

16. Zadania do samodzielnej realizacji

1. Programy (obowiązkowe)
   - **Zadanie 1**: Napisz podprogram odmierzający czas 104us. Zastosuj napisany podprogram w programie programowego nadajnika UART. Uwzględnij opóźnienia spowodowane przez rozkazy CALL i RET.
   - **Zadanie 2**: Napisz program, który przez port szeregowy wyśle twoje imię i nazwisko zapisane znakami w kodzie ASCII.
   - **Zadanie 3**: Napisz program, który uaktywni sprzętowy odbiornik i nadajnik modułu UART dla parametrów 57600,8,N,1. Program czeka na odebranie bajtu i natychmiast go wysyła przez port szeregowy. Częstotliwość zegara procesora 8 MHz.

3. Zadania dodatkowe (dla chętnych)
   - **Zadanie A**: Napisz program, który wyśle przez UART (programowy) zawartość 16 kolejnych komórek pamięci danych SRAM (od adresu 0x0060).

17. Sprawozdanie

- W sprawozdaniu należy umieścić kody wykonanych programów z odpowiednim wyjaśnieniem działania zastosowanych dyrektyw i instrukcji.
- Na podstawie noty katalogowej ATmega16 wyjaśnić, dlaczego w prezentowanych programach wykonano skok do adresu 0x0030, omijając pewien obszar pamięci programu (*strona 45, Interrupts*).